01 / 先看结论与边界

从 ESP32 出门断网，到私有地址、路由与叠加网络

同是 192.168，不一定在一个网

100.90.0.1 不是 RFC1918 私有地址

13 个问题 · 6 步主实验 · 真实跨网转发与用户态 WireGuard

已实读原帖全文、对话回复，并抽帧看完 11 秒附件视频。

作者是 @0xNathanWalk。主帖发表于 2026-10-20 21:39:35（Asia/Shanghai）。正文里的网络例子可以逐条用 RFC 与程序核对；作者提到的 Stopclock 新固件与“ESP32 也能玩 Tailscale”，没有在对话里给出仓库或拓扑图。[S1]

> **状态：学习交付完成 / ESP32 实机与 Tailscale 登录待验证**
> 云端已真实运行：Python 子网判定与最长前缀匹配、本机 `ip addr/ip route` 只读快照、user namespace 里两个 /24 加路由器的跨网转发正负对照，以及 wireguard-go（SHA `b3bc704`）在两命名空间之间的加密隧道。没有登录 Tailscale 账号，没有使用用户密钥，没有改宿主机全局转发或安全设置。没有 ESP32 实机，也没有刷作者未公开的固件。

查重固定 reminder_knowlege main `a8674777e18302b8804b137212b1396cbfe5a747`（2026-11-04 17:42:58 Asia/Shanghai）。总览与相关路径中没有 RFC1918、DHCP、子网掩码、Tailscale 或 ESP32 联网专条；已有的“网关/路由”笔记属于 API/技能路由，不能合并。

由浅入深 / 地址与子网

Q01 两家都能用 192.168.1.20，为什么还是连不上？

因为私有地址只在*当前这张网*里有意义。RFC1918 为私有互联网保留三段：10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。不同家庭、公司都可以复用这些号码，它们并不是全球唯一标识。[S2 §3]



> 作者的例子是对的。本批实验里的云主机地址 172.30.0.2 也落在 172.16.0.0/12 里，但那是这台云电脑的底层网，不是作者家里的局域网，不能拿来“复现他家的 192.168.1.0/24”。

Q02 子网掩码到底划出了哪一段？

把 IP 和掩码做按位与，就得到网络前缀。192.168.1.20 加上 255.255.255.0，等于 192.168.1.0/24；/24 是同一掩码的 CIDR 写法。该网共 256 个地址：.0 是网络号，.255 是广播，通常可分配 .1–.254 共 254 个，网关再占其中一个，还剩 253 个。[S1；E2]



> 原帖“当腰访问”是笔误，按“当要访问”理解。程序核对：.1 到 .254 的可分配区间与帖文一致。

Q03 目标也是私有地址，为什么有时还要走网关？

主机先看目标是否落在*自己这张*子网里。192.168.1.20/24 访问 192.168.1.30 走本地直达；访问 192.168.2.30 虽然也是 RFC1918，但不在 192.168.1.0/24，应交给默认网关转发。私有只说明地址类别，不说明“和我在同一二层网”。[S1；E2]



由浅入深 / 切分、DHCP、路由表

Q04 /25 切开以后，.20 和 .150 为什么不是一家？

掩码可以相同，地址范围必须说清楚。两个 /24（192.168.1.0/24 与 192.168.2.0/24）前缀不同；把一个 /24 对半切成 /25 后，192.168.1.0/25 覆盖 .0–.127，192.168.1.128/25 覆盖 .128–.255。.20 与 .150 前三段相同，仍分属两网。路由查表时更长的前缀优先，所以若同时存在 /24 与 /25，.150 仍应命中 /25。[S1；E2；E3]



Q05 IP 和网关是谁发的？DHCP 要走几步？

家里设备通常用 DHCP 自动拿到 IP、掩码、网关、DNS 和租约。RFC2131 新地址分配的四次握手是：DHCPDISCOVER → DHCPOFFER → DHCPREQUEST → DHCPACK。[S3 表 2 与图 3]



> 教学状态机（E2）里，把 192.168.1.30 做成手机的地址保留后，手机仍走完整四步，只是每次拿到指定地址；另一台设备从地址池领到 192.168.1.100。这不是在主机上手工填写的静态 IP。本批没有在宿主机网卡上运行真实 DHCP 客户端。

Q06 网关一定是 192.168.1.1 吗？

不是。这只是常见习惯，也可以是 .254 或其他可用地址，前提是路由器真的用这个地址。手工静态配置，与路由器里的 DHCP 地址保留，结果看起来都“永远是这个号”，但后者仍通过 DHCP 获取。[S1]



Q07 路由表是怎样“选路”的？

对每个目标，找出所有覆盖它的表项，选前缀最长的那一条；什么都配不上就不达。本批用帖中例子加一条 100.64.0.0/10 的叠加路由做查表：192.168.1.30 走局域网，192.168.2.30 与 8.8.8.8 走默认网关，100.90.0.1 走叠加接口。[E3]



深入一层 / 出门为什么会挂，Tailscale 又补了什么

Q08 为什么 ESP32 在家里能上网、出门就挂?

常见原因至少有两层。第一，站点模式要先关联到一份已配置的 SSID；只写了家里 Wi-Fi，就关联不上外面的网，DHCP 也无从开始。作者说已把手机热点写进固件，并打算继续加入常去的网络，针对的就是这一层。[S1] 第二，即便在外面另连上一张网，家里 Mac mini 的 192.168.1.30 仍是私有地址，图书馆的网不会把它当本地设备，默认也无法从公网直接拨到这个号码。要访问家里的服务，需要叠加地址、端口映射、或中继，而不是“还记得那个私有 IP”就够了。



> ESP-IDF 的 STA 示例通过 ESP-NETIF 使用 lwIP，接口对象里可启用 DHCP 客户端；连接上 AP 后由 DHCP 填写 IP。本批阅读了 v6.1 的 ESP-NETIF 文档与 `esp_netif.h`，没有在实机或模拟器里跑 STA。[S7]

Q09 Tailscale 地址为什么换网络也不变？它是私有 IP 吗?

官方文档写明：Tailscale 给 tailnet 中每台设备分配一个来自 RFC6598 共享地址空间（常称 CGNAT）的地址，范围是 100.64.0.0/10（100.64.0.0–100.127.255.255）。设备换 Wi-Fi 或蜂窝网时，这个地址保持不变。作者举的 100.90.0.1 落在该段内，程序判定为 RFC6598，**不是** RFC1918 私有地址。[S4；S5；E2]



> RFC6598 开篇就写：共享地址空间与 RFC1918 私有地址不同，原本预留给运营商 CGN。Tailscale 选用它的原因之一，是减少和家里常见的 10/8、192.168/16 冲突；若与其他也占用 100.64/10 的 VPN 同时开启，仍可能冲突。[S4；S5]

Q10 “能直连就直连，否则中继”——外层数据到底经过谁？

Tailscale 的数据面建立在 WireGuard（官方说明是用户态 Go 实现 wireguard-go）之上；控制面负责交换密钥与端点。直连走 UDP 打洞；打不通时用 DERP（Designated Encrypted Relay for Packets）这类中继。图书馆路由器转发的是封装后的外层包，看不到内层明文。[S6]



> E5 用两个命名空间模拟“不同底层子网 + 加密叠加”：访问 10.66.0.2 成功的同时，路由器抓到 13 个 UDP/51820、0 个 TCP、0 个 ICMP。这是本批教学隧道的实测，不是对 Tailscale DERP 或协调服务器的实测。

深入一层 / 证据边界与用途

Q11 ESP32 上到底有没有 Tailscale 客户端？

要分层看，不能合成一句“已经官方支持”。官方“更小二进制”指南面向 Linux 类嵌入式，把合并后的 Go 二进制压到约数兆字节量级，并写明这不是为 ESP32 准备的；官方仓库 issue #5220 至 2026-06 仍开放，讨论的是社区移植。[S8]

社区有 CamM2325/microlink 等 ESP-IDF 客户端（本批固定 `216da330`），自称静态 SRAM 约 85–116 KB，并建议 ESP32-S3 + PSRAM。这不是官方产品，也不是作者仓库。[S9]

作者对“esp32有tailscale？”回“嗯”，对“轻量客户端会爆内存吗”回“不会”，但又对“请补项目/固件链接和哪台设备跑 Tailscale”回“回头我写个详细教程”。后续他还写过“手机没用 Tailscale，用小火箭给 ESP32 做热点”“装盒 Tailscale 可以随身携带，连手机 Wi-Fi”。拓扑因此**不能唯一确定**：可能是板子自己跑客户端，也可能是旁边一个 Tailscale 盒子负责叠加，ESP32 只是通过热点上网。本批不把作者口头确认写成已核实的开源仓库。[S1]



Q12 “眨眼=有网，闭眼=断网”已经证实了吗？

没有。这是作者对新固件的说明。评论问眨眼判断的是关联上 Wi-Fi，还是家里的服务真的有响应；作者没有回答。附件 11.18 秒视频拍的是车内手持圆形设备（外形与 M5Stack StopWatch 文档中的圆形 AMOLED 套件相符），表盘是黄色头像、数字 16,929、约 89% 电量和 Wi-Fi 图标。抽帧未见“闭眼断网”对照，也不能把“有 Wi-Fi 图标”等同于“已经连上家里的服务或 tailnet”。[S1；S10]



> M5Stack 官方 StopWatch 为 ESP32-S3R8、16 MB Flash、8 MB PSRAM。作者混用 Stopclock/StopWatch 名称。官方出厂固件是表盘、秒表、徽章等，不是本帖描述的眨眼联网固件。实机验证待做。[S10]

Q13 叠加地址会不会和家里的网撞车？

与常见 RFC1918 网段冲突的概率较低，这是官方给出的设计理由之一。但 100.64.0.0/10 并不是“可以随便当第二张局域网用”的私有段；它仍可能和其他占用同一段的 VPN 冲突。本批教学隧道刻意使用 10.66.0.0/24，避免让人误以为我们分配了真实的 Tailscale 地址。



可以马上用到的五件事

- 解释“两台设备都是 192.168.x 却不通”：先画各自的前缀，再看要不要网关，而不是只看是不是私有地址。
- 读懂一台电脑的 `ip addr` 与 `ip route`：哪条是直连，哪条是默认网关，哪条是叠加网。
- 规划出门访问家里的设备时，把“底层 IP / SSID”和“叠加 IP / 稳定标识”分成两列，避免把 192.168.1.30 写进外出客户端。
- 给 ESP32 或多网络设备列 SSID 清单（家、热点、常去的网）时，同时写清：关联成功、DHCP 成功、叠加或业务探测成功，是三个不同的灯。
- 在没有实机、也不登录商业 VPN 的前提下，用两个网络命名空间验证“不转发则跨网失败、转发则成功”，再用用户态 WireGuard 看清外层 UDP 与内层业务的区别。

03 / 主实验：从查表到真转发

在解压 exercise ZIP 后进入该目录。E1–E3、E6 只需 Python 3 标准库。E4–E5 需要 `ip`，并在 user namespace 内搭建临时拓扑；脚本结束即清理，不写宿主机 `ip_forward`。本云端没有 `ping`，跨网探测用 Python TCP echo。

E1 环境与本机路由快照（只读）

目的：先固定实验发生在哪台机器上，避免把云电脑的网段写成作者的家网。

```
python3 lab.py env
```

真实结果：Python 3.13.5，时间 2026-11-11 01:18:34（Asia/Shanghai）。`ip_forward=1`（只读，未改）。接口：lo 127.0.0.1/8；enp0s3 172.30.0.2/24，默认路由经 172.30.0.1；docker0 172.17.0.1/16（linkdown）。两段都在 RFC1918 的 172.16.0.0/12 内。

验证：对照 `ip -br addr` 与 `ip route` 原文；不把 172.30.0.0/24 解释成帖中的 192.168.1.0/24。

E2 子网判定、地址分类、DHCP 状态机

目的：用标准库 `ipaddress` 逐条核对原帖例子，并区分 RFC1918 与 RFC6598。

```
python3 lab.py subnet
```

真实结果：9 条成员关系全部符合预期，包括 /25 切开后 .20 与 .150 分家，以及 100.90.0.1 ∈ 100.64.0.0/10 且分类为 RFC6598-CGNAT-shared。192.168.1.0/24 共 256 址，可分配 .1–.254，若网关占 1 个则余 253。DHCP 模拟：保留地址的“手机”经四步拿到 192.168.1.30，另一设备从池中拿到 192.168.1.100。

验证：程序对每条断言 `member == expected`；分类函数把 8.8.8.8 标为 public-or-other。这不是真实 DHCP 数据包。

E3 最长前缀匹配

目的：在“局域网 /24 + 叠加 100.64/10 + 默认路由”这张表上查帖中的目标。

```
python3 lab.py route
```

真实结果：192.168.1.30 与 192.168.1.150 命中 connected-lan；192.168.2.30、8.8.8.8、10.0.0.1 命中默认网关；100.90.0.1 命中 overlay-cgnat。/25 切分表中 .20 走 office、.150 走 guest；即使再加入一条 192.168.1.0/24，.150 仍因 /25 更长而走 guest。
验证：每条查找都有断言。这是用户态查表，不是内核 FIB 的完整复刻。

03 / 主实验：隔离拓扑里的真包

E4 两个 /24 + 路由器：转发关则失败，开则成功

目的：复现“目标也是私有地址，但仍要网关转发”。办公网 192.168.10.20/24，访客网 192.168.20.30/24，路由器 192.168.10.1 与 192.168.20.1。全部放在 user+mount+net 隔离命名空间，不改宿主机接口。

```
bash netns_lab.sh   # 内含 E4 与 E5；需 ip 与 Python
```

真实结果（最终记录 2026-11-11 01:19:22 Asia/Shanghai）：路由器转发=0 时，办公网访问访客网 :9000 为 TimeoutError，1003.8 ms。路由器自己访问访客网成功（3.2 ms），说明服务是活的，只是没跨网。写入路由器 netns 的 ip_forward=1 后（仅该 netns），办公网→访客网 3.6 ms 成功，反向 4.4 ms 成功。宿主机 ip_forward 实验前后都是 1，接口仍为 lo/enp0s3/docker0，命名空间退出后列表为空。
验证：必须同时看到“关：超时失败 / 开：双向成功 / 宿主机接口与转发值不变”。本探测是 TCP echo，不是 ICMP。

E5 用户态 WireGuard 叠加隧道

目的：在已经能转发的底层之上，用官方开源 wireguard-go（`b3bc704ef0ee21a36298601a9e1e5ff90377ac52`）打一条 10.66.0.1 ↔ 10.66.0.2 的加密隧道。临时密钥只在隔离目录生成并删除，日志只留公钥。不登录 Tailscale。 **真实结果：**两端 UAPI `errno=0`。办公网访问 10.66.0.2:9100 成功，5.4 ms，载荷 `PONG:PING`。路由器在 r-o 上抓包 4 秒：13 个 UDP/51820（1842 字节），0 个 TCP，0 个 ICMP；样本为 192.168.10.20 ↔ 192.168.20.30 的 51820 端口。内核提示“已有一等 WireGuard 支持，不必跑用户态”，本批仍按任务使用用户态进程，没有改用 `ip link add type wireguard`。

验证：叠加 TCP 成功，且路由器看不到内层 TCP，只看到外层 UDP。这不是 Tailscale 控制面、DERP 或 NAT 打洞的实测。首次尝试曾因两个 netns 共享 /run 且都叫 wg0 导致套接字占用，已改为 wga/wgb 后重跑；失败过程留在实验日志中。

E6 重放纯函数结果

```
python3 lab.py all    # 含 verify；或 python3 lab.py verify
```

真实结果：E2/E3 自检 `ok: true`，results.json 已落盘。

验证：这是同一份教学代码自己复核，不是独立审核，也不覆盖 E4/E5 的网络命名空间（那两步以 netns-results.json 为准）。

待验证（明确未做）：ESP32/StopWatch 实机刷写与多 SSID；作者未公开固件的眨眼规则；登录 Tailscale / 使用预授权密钥；官方协调服务器与 DERP 路径；ESP-IDF 模拟器里的 STA+DHCP。不要把 5.4 ms 的教学隧道时延说成 Tailscale 性能。

04 / 合入候选与来源固定

增量，而不是改写旧笔记里的“网关”

已有概念如「统一数据网关」「技能路由器与授权硬门」「能力层与后端路由」讨论的是 API/技能，不是 IPv4 转发。建议新增：私有地址与复用、子网掩码与 CIDR、默认网关与跨网转发、DHCP 租约与保留、最长前缀匹配、RFC6598/CGNAT 与 Tailscale 地址、叠加网络与中继（概念）。互链即可，不要合并到旧“网关”条目。

来源	固定版本与范围
原帖与对话	2112539228278239708，2026-10-20 21:39:35 Asia/Shanghai；对话已读至次页 0 条。视频已下载并抽帧，未完整声称看清眨眼动画。
RFC1918 / 2131 / 6598	rfc-editor 文本。1918 §3 三段私有块；2131 表 2 与图 3 四步握手；6598 规定 100.64.0.0/10，并写明有别于 RFC1918。
Tailscale 文档与仓库	How it works（2020-03-20）；100.x 文档（页面标注 Last validated Jan 12, 2026）；小二进制指南（Jan 5, 2026）。仓库 tip 读取为 05930082…（2026-11-11 00:56:08 Asia/Shanghai），仅作当时快照，正文论证不依赖这一笔提交。
wireguard-go	b3bc704ef0ee21a36298601a9e1e5ff90377ac52，2026-10-25 06:09:52 Asia/Shanghai。E5 实际编译并运行。
ESP-IDF / M5Stack / 社区客户端	ESP-IDF v6.1 标签对象指向 commit fff9895c…，标注 2026-08-25 11:21:22 Asia/Shanghai；阅读 ESP-NETIF 文档与 esp_netif.h。M5Stack StopWatch 官方文档。microlink 固定 216da330…（2026-03-17 21:06:57 Asia/Shanghai），仅作社区对照，不是作者项目。

来源短码

- [S1] https://x.com/0xNathanWalk/status/2112539228278239708 及同对话作者回复；后续拓扑线索帖 2115089119001170072、2115837426581348694
- [S2] https://www.rfc-editor.org/rfc/rfc1918.txt
- [S3] https://www.rfc-editor.org/rfc/rfc2131.txt
- [S4] https://www.rfc-editor.org/rfc/rfc6598.txt
- [S5] https://tailscale.com/kb/1015/100.x-addresses （Last validated Jan 12, 2026）
- [S6] https://tailscale.com/blog/how-tailscale-works
- [S7] ESP-IDF v6.1 ESP-NETIF 与 components/esp_netif/include/esp_netif.h（commit fff9895c82d744c7237be8847347bdd1b07c6643）
- [S8] https://tailscale.com/docs/how-to/set-up-small-tailscale ；https://github.com/tailscale/tailscale/issues/5220
- [S9] https://github.com/CamM2325/microlink/tree/216da3300f0493b0860247d43f7af5ce29df63a5
- [S10] https://docs.m5stack.com/en/core/StopWatch

详细 URL、文件指纹、查重与合入建议见 sources.md、knowledge-proposal.md；完整真实输出见 experiment-log.md。本文由同一 worker 自查，不声称独立审核。未更改共享 vault、用户任务状态或日程。未购买服务，未使用用户密钥。